Chapitre 2 : Structures de Données Dynamiques (Listes, Piles et Files)

IN323 - Programmation en langage C++

Elisa HENRION-GUENEAU

IPSA - Site Toulouse

AERO3 - 2025/2026

Structures récursives

Définition

Structure récursive

Une structure récursive est une structure dont au moins l'un des composants permet de désigner une autre instance de cette structure.

Syntaxe en C

```
typedef struct Cellule {
    int val;
    struct Cellule *suivant; // Pointeur recursif
} Cellule;
```

Définition

Syntaxe en C

```
typedef struct Cellule {
    int val;
    struct Cellule *suivant; // Pointeur recursif
} Cellule;
```

Attention :

- Le `struct Cellule` au début : nécessaire pour que la structure puisse se pointer elle-même.
- Le `Cellule` à la fin : c'est un alias pour se servir de la structure plus facilement dans le reste du programme.

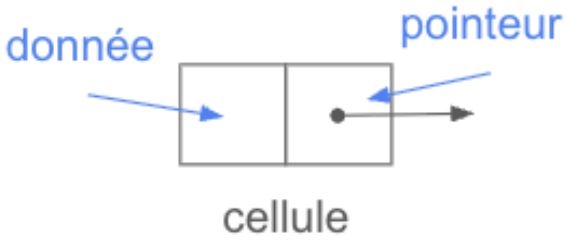
Listes chaînées

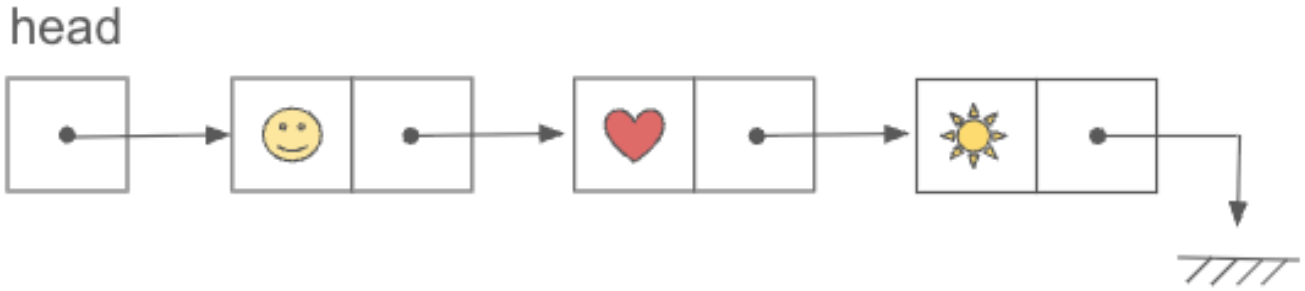
Listes simplement chaînées

Définition

Liste simplement chaînée ⇔ suite de cellules
Chaque cellule désigne la suivante grâce à un pointeur.

Cellule = Valeur + Pointeur vers la suivante.





Avantages et Inconvénients

Avantages

- Les cellules ne sont pas stockées de manière contiguë en mémoire.
- Taille dynamique.
- Insertion/Suppression facile sans décaler les éléments.

Inconvénients

- Pas d'accès direct (ex : `tab[i]` impossible).
- Surcoût mémoire (stockage des pointeurs).

Implémentation en C

Définition des types :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Cellule {
    int val;
    struct Cellule *suivant;
} Cellule;

typedef struct {
    Cellule *head; // NULL par defaut
} Liste;
```

Implémentation en C

Affichage d'une liste chaînée

```
void afficher(Liste l) {
    Cellule *c = l.head;
    while (c != NULL) {
        printf("%d -> ", c->val);
        c = c->suivant;
    }
    printf("\n");
}
```

Opérations sur les listes chaînées

- Tester si la liste est vide (**isEmpty**)
- Renvoyer la longueur de la liste (**length**)
- Ajouter un élément au début (**cons**)
- Ajouter un élément à la fin (**append**)
- Renvoyer l'élément de tête (**head**)
- Rechercher un élément (**search**)
- Supprimer un élément (**removeElt**)
- Convertir un tableau en liste (**fromArray**)
- Libérer la mémoire (**clear**)

Exemple d'utilisation :

```
Liste l = {NULL};
cons(&l, 10);     // Liste : 10 -> NULL
append(&l, 20);   // Liste : 10 -> 20 -> NULL
cons(&l, 5);      // Liste : 5 -> 10 -> 20 -> NULL
afficher(l); // Affiche 5 10 20

Cellule *trouve = search(l, 10);
if (trouve != NULL) {
    printf("Valeur %d trouvee a l'adresse %p\n",
        trouve->val, trouve);
}
removeElt(&l, 10);
afficher(l); // Affiche 5 20
clear(&l);
```

Opérations sur le type Liste

Fonction isEmpty : Est-ce que la liste est vide ?

Opérations sur le type Liste

Fonction length : Quelle est la taille de la liste ?

Il faut forcément parcourir toutes les cellules de la liste...

Opérations sur le type Liste

Fonction cons : Ajouter en tête

On passe un pointeur vers la liste pour modifier sa `head`.

Opérations sur le type Liste

Fonction head : Renvoyer le 1er élément

On doit gérer le cas où la liste est vide
→ Sortir avec code erreur → exit(1)

Opérations sur le type Liste

Fonction append : Ajouter à la fin

Il faut créer une nouveau cellule et l'ajouter à la fin de la liste (ou en tête si la liste est vide)

Opérations sur le type Liste

Fonction clear (Indispensable !) : libérer chaque cellule manuellement.

Opérations sur le type Liste

Le reste à faire en TD ...

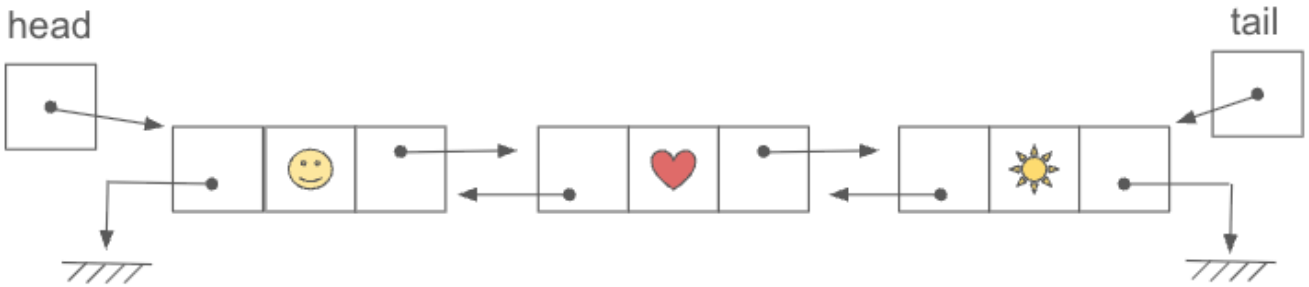
Listes doublement chaînées

Il existe aussi des liste doublement chaînées

Définition :

Cellule = valeur + pointeur vers cellule précédente + pointeur vers cellule suivante

→ Permet un parcours bidirectionnel



Implémentation en C

Définition des types :

```
typedef struct CelluleD {
    int val;
    struct CelluleD *suivant;
    struct CelluleD *precedent;
} CelluleD;

typedef struct {
    CelluleD *head;
    CelluleD *tail;
} ListeD;
```

Piles

Présentation

Définition :

Une pile est une structure de données avec un seul point d'accès : son **sommet**.

- On insère les données sur le sommet (on les "**empile**").
- On extrait les données depuis le sommet (on les "**dépile**").

Structure LIFO :

C'est une structure **LIFO (Last In First Out)** : le dernier élément ajouté sera le 1er traité

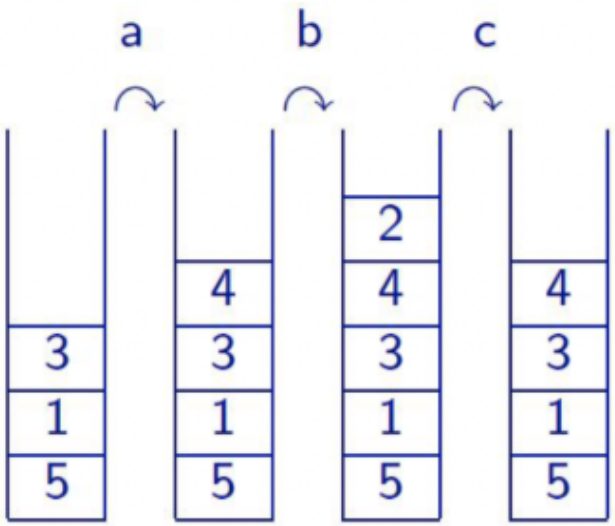
Rq : Structure très utilisée en informatique (variables locales et résultats de fonctions, par exemple, sont créés sur une pile)

Présentation

a. insérer 4

b. insérer 2

c. extraire



Opérations sur les piles

- **empiler** et **depiler**
- **creerPile** : Initialiser une pile vide
- **estPileVide** : Tester si une pile est vide
- **sommet** : Accéder à la valeur du sommet sans la retirer.
- **clearPile** : Libérer toute la mémoire de la pile.
- On peut aussi ajouter des fonctions de conversion :
  - **fromArray** : Tableau $\rightarrow$ Pile
  - **toArray** : Pile $\rightarrow$ Tableau

Rq :

- **depiler** et **sommet** doivent gérer le cas de la pile vide.
- Il est souvent pratique d'écrire **Depiler** pour qu'elle revoie l'élément dépilé.

Implémentation

Le type Pile

Pour créer une pile on utilise la structure `Cellule`, et on crée une nouvelle pour la Pile.

```
typedef struct {
    Cellule *sommet; // Pointeur vers le haut de la
        pile
} Pile;

Pile creerPile() {
    Pile p = {NULL};
    return p;
}
```

Implémentation

```
int main() {
    Pile p = creerPile();
    empiler(&p, 5);
    empiler(&p, 10);
    empiler(&p, 15);

    printf("On retire sommet : %d\n",depiler(&p));
    printf("Nouveau sommet : %d\n",sommet(p));

    while (!estPileVide(p)) {
        printf("%d \n", depiler(&p));
    }
    clearPile(&p);
    return 0;
}
```

Files

Présentation

Définition :

Une file est une structure de données avec deux points d'accès : **head (tête)** et **tail (queue)**.

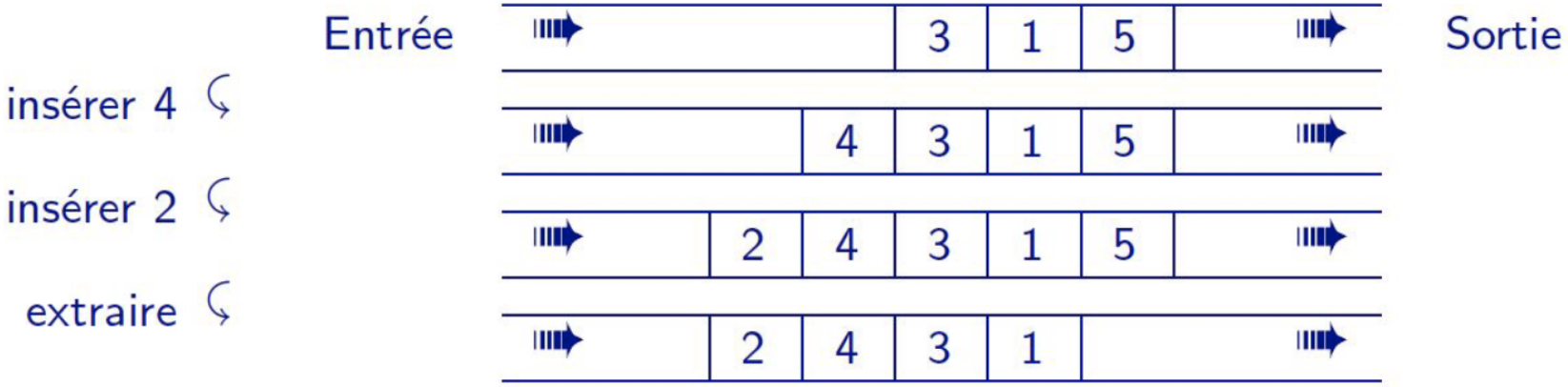
- On insère les données dans la queue de la file (on les "**enfile**").
- On extrait les données depuis la tête (on les "**défile**").

Structure FIFO :

C'est une structure **FIFO (First In First Out)** : le 1er élément ajouté sera le 1er traité

Rq : Structure très utilisée en informatique (file d'attente de processus, de travaux d'impression, etc...)

Présentation



Opérations sur les files

- **enfiler** et **defiler**
- **creerFile** : Créer une file vide
- **estFileVide** : Tester si une file est vide
- **head** : Accéder à la valeur en tête de file sans la retirer
- **clearFile** : Libérer toute la mémoire de la file.
- On peut aussi ajouter des fonctions de conversion :
  - **fromArray** : Tableau $\rightarrow$ File
  - **toArray** : File $\rightarrow$ Tableau

Rq :

- **defiler** et **head** doivent gérer le cas de la file vide.
- Il est souvent pratique d'écrire **defiler** pour qu'elle revoie l'élément défilé.

Implémentation

Le type File

Pour être efficace, la File utilise deux pointeurs vers des structure `CelluleD` (liste doublement chaînée)

```
typedef struct {
    CelluleD *head; // Sortie de la file
    CelluleD *tail; // Entree de la file
} File;

File creerFile() {
    File f = {NULL, NULL};
    return f;
}
```

Implémentation

```
int main() {
    File f = creerFile();
    enfiler(&f, 1);
    enfiler(&f, 10);
    enfiler(&f, 20);

    printf("On defile : %d\n",defiler(&f));
    printf("Nouvelle tete : %d\n",head(f));
    while (!estFileVide(f)) {
        printf(" %d\n", defiler(&f));
    }
    clearFile(&f);
    return 0;
}
```

Une file particulière : File à priorités

FAP (File à priorité) :

- Chaque élément de la FAP est muni d'une priorité (un entier)
- L'élément de priorité max (entier min) est le premier traité

→ **A faire en TD**

Rq : Structure utilisée en informatique, quand le système d'exploitation doit gérer des processus.